

北京理工大学

人工智能基础项目报告

基于 PyTorch 的小型 GPT 模型构建与训练

Building and Training a Small GPT Model with PyTorch

学 院： 自动化学院

专 业： 自动化

学生姓名： 姜尹雯

学 号： 1120233551

指导教师： 周志强

2026 年 6 月 7 日

目 录

第 1 章	绪论	1
1.1	项目背景与意义	1
1.2	项目任务与技术路线	1
第 2 章	大语言模型相关理论基础与关键知识点	3
2.1	GPT 类语言模型基本原理	3
2.2	文本分词与输入表示	4
2.3	Transformer 核心机制	4
2.4	预训练与微调机制	5
第 3 章	小型 GPT 模型设计与实现	6
3.1	模型总体结构	6
3.2	核心模块设计	7
3.2.1	MiniGPTConfig 配置类	7
3.2.2	因果多头自注意力模块	7
3.2.3	前馈网络与 Transformer 模块	8
3.2.4	MiniGPT 模型主体	8
3.3	模型参数配置与低算力适配	9
第 4 章	训练数据准备与模型预训练实验	10
4.1	TinyStories 数据集与文本预处理	10
4.2	分词编码与训练样本构造	11
4.3	预训练策略与程序实现	12
4.4	预训练结果与生成效果分析	13
第 5 章	微调实验与任务适应	15
5.1	微调任务与监督数据构建	15
5.2	微调策略与程序设计	17
5.3	微调结果与任务效果分析	18
5.4	本章小结	21
5.5	本章小结	22
第 6 章	总结与体会	22
参考文献与开源资源说明		24
A	项目文件结构	25
B	主要运行命令	26
C	主要输出文件	26

D	实验结果截图清单	27
---	--------------------	----

第 1 章 绪论

1.1 项目背景与意义

近年来，以 GPT 为代表的大语言模型在自然语言处理、智能问答、文本生成、代码生成等领域得到了广泛应用。大语言模型通常基于 Transformer 结构，通过大规模文本数据训练，使模型能够学习语言中的上下文关系和文本生成规律。与传统自然语言处理方法相比，大语言模型具有更强的通用性和迁移能力，已经成为当前人工智能领域的重要研究方向之一。

本项目以“从零构建大模型”为主题，重点并不是训练一个真正具有大规模参数量和复杂能力的商用模型，而是在低算力平台上实现一个小参数量 GPT 类语言模型。通过完成模型搭建、文本数据处理、预训练、微调和推理展示等过程，可以更加直观地理解大语言模型的基本结构和工作原理，从实践角度加深对 Transformer、注意力机制、语言建模以及微调方法的认识。

1.2 项目任务与技术路线

根据课程作业要求，本项目需要围绕“从零构建大模型”的完整流程展开。首先，需要梳理大语言模型构建过程中涉及的重要知识点，包括 GPT 类模型的基本架构、模型参数配置、文本数据准备、分词方法、词嵌入、位置编码、预训练目标以及微调方法等内容。其次，需要基于 PyTorch 搭建一个小参数量 GPT 类语言模型，使其能够在普通单机或较低算力平台上完成训练和推理，并体现大语言模型中的核心机制。然后，需要准备小规模文本语料，完成文本编码、训练样本构造和预训练程序设计，使模型能够通过预测下一个 token 的方式学习基本语言规律。在此基础上，还需要构建监督微调数据，选择合适的微调任务和训练策略，使预训练后的模型能够适应特定任务。最后，需要对实验结果进行展示和分析，总结项目完成情况、遇到的

问题以及学习收获。

本项目的整体技术路线可以概括为以下几个步骤：

1. 数据准备与分词编码：选取 Hugging Face 平台上的 TinyStories 数据集作为预训练语料，对文本进行清洗和拼接，并采用基于正则表达式的分词器将文本转换为 token id 序列；
2. 训练样本构造：利用滑动窗口方法将 token 序列切分为固定长度的输入序列，并将其右移一位作为预测目标，构成 next-token prediction 训练样本；
3. 小型 GPT 模型搭建：基于 PyTorch 构建小参数量 GPT 类模型，主要包括词嵌入、位置嵌入、因果多头自注意力、前馈网络、残差连接和层归一化等模块；
4. 预训练与生成测试：使用 TinyStories 样本对模型进行预训练，通过交叉熵损失优化下一个 token 的预测结果，并在训练后根据给定 prompt 进行自回归文本生成；
5. 微调与结果分析：在预训练模型基础上构建简单监督任务进行微调，并结合 loss 曲线、perplexity、生成样例和微调结果，对模型效果进行分析总结。

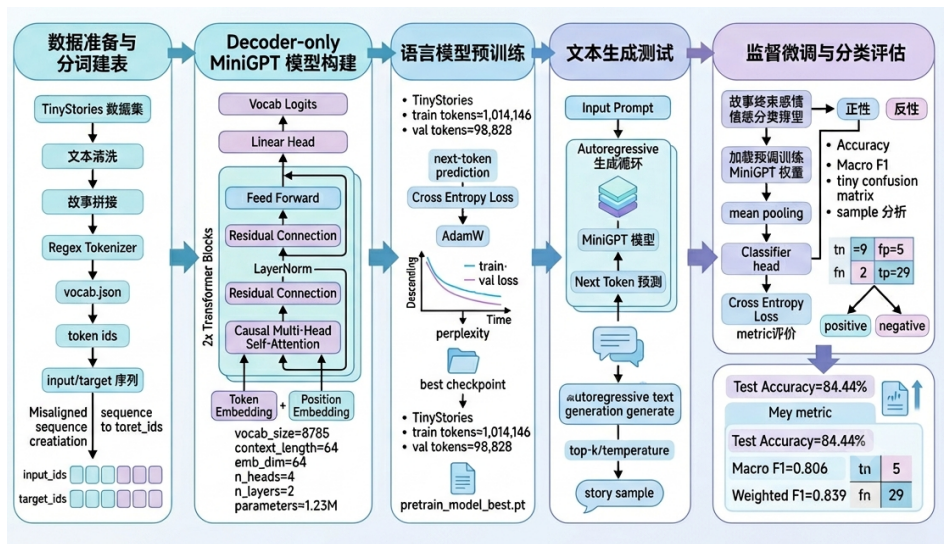


图 1-1 MiniGPT 项目技术路线图

第 2 章 大语言模型相关理论基础与关键知识点

2.1 GPT 类语言模型基本原理

自回归建模思想。 GPT 类语言模型的核心任务是根据已有上下文预测下一个 token。文本经过分词后可表示为 token 序列 $x_1, x_2, \dots, x_T$ ，模型并不是一次性生成完整文本，而是按照从左到右的顺序逐步建模当前位置的条件概率。对应到训练阶段，next-token prediction 要求模型根据前 t 个 token 预测第 $t + 1$ 个 token，其语言建模目标可写为：

$$P(x_1, x_2, \dots, x_T) = \prod_{t=1}^T P(x_t | x_{<t}),$$

$$\mathcal{L}_{\text{LM}} = -\frac{1}{T-1} \sum_{t=1}^{T-1} \log P(x_{t+1} | x_{\leq t}).$$
(2-1)

式中，第一行表示自回归概率分解，第二行表示下一个 token 预测任务中的平均交叉熵损失。

Decoder-only 结构。为了服务于从左到右的生成目标，GPT 采用 Decoder-only Transformer 结构，并通过因果遮罩限制每个位置只能使用当前位置及其之前的信息。这样，模型在训练时不会提前看到未来 token，在推理时也可以不断把新生成的 token 拼接回上下文，继续预测后续内容。本文实现的 MiniGPT 是该结构的小型化版本，保留词嵌入（token embedding）、位置嵌入（position embedding）、因果自注意力、前馈网络、残差连接和 LayerNorm 等关键组件，但通过较小的层数、嵌入维度和上下文长度降低训练成本。图 2-1 给出了大语言模型从文本输入到模型输出的整体结构关系，可作为后续 MiniGPT 实现流程的概念参考。

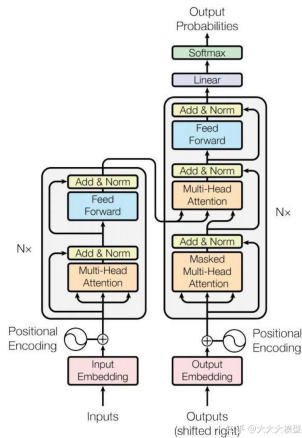


图 2-1 大语言模型整体结构示意图

2.2 文本分词与输入表示

分词器的作用。神经网络不能直接处理原始字符串，因此文本需要先经过分词器（tokenizer）转换为 token，再映射为整数形式的 token id。分词器主要完成文本切分、词表构建、编码和解码：编码将文本转为模型输入，解码则将模型生成的 token id 还原为可读文本。本文保留 SimpleRegexTokenizer 和 GPT2Tokenizer 两种实现，前者面向低算力小模型实验，后者作为更接近真实 GPT 系列子词分词方式的可选方案。预训练语料中使用 `<|endoftext|>` 作为故事边界标记，具体数据拼接、缓存、分词器对比和样本构造过程放在第四章说明。

输入表示。得到 token id 后，模型还需要将离散编号映射为连续向量。GPT 的输入表示由词嵌入和位置嵌入相加得到：

$$\mathbf{h}_0 = \mathbf{E}_{\text{tok}}(\mathbf{x}) + \mathbf{E}_{\text{pos}}(\mathbf{p}). \quad (2-2)$$

其中， $\mathbf{x}$ 表示 token id 序列， $\mathbf{p}$ 表示位置编号序列。词嵌入提供 token 本身的语义表示，位置嵌入提供顺序信息，二者共同构成 Transformer 模块的输入。

2.3 Transformer 核心机制

Transformer 模块的作用。GPT 的主体由多层 Transformer 模块堆叠而成。Transformer 模块通过因果自注意力建模上下文关系，再通过前馈网络进行非线性变换，并

使用残差连接和 LayerNorm 稳定训练。对本文项目而言，需要重点理解的是：自注意力让模型能够从上下文中选择相关信息，因果遮罩保证模型不能看到未来 token，多层堆叠则逐步形成更高层次的文本表示。图 2-2 展示了 Transformer 模块中注意力、前馈网络以及残差归一化结构之间的关系。

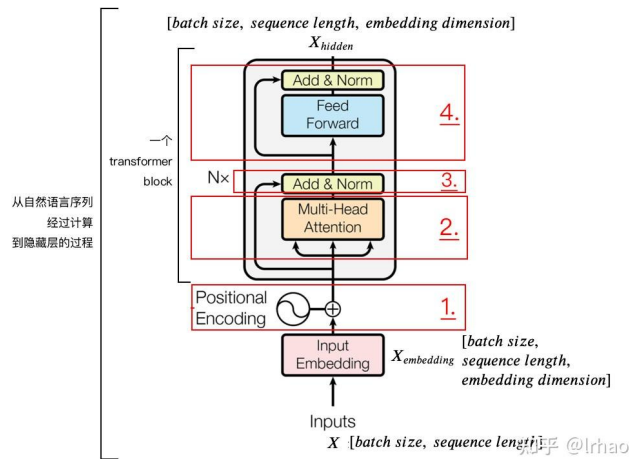


图 2-2 Transformer 核心结构示意图

因果自注意力机制。注意力计算通常由 Query、Key 和 Value 三组向量完成。在 GPT 中，为了满足自回归生成约束，需要在注意力分数中加入因果遮罩矩阵 $\mathbf{M}$ ：

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T + \mathbf{M}}{\sqrt{d_k}} \right) \mathbf{V}. \quad (2-3)$$

其中， $\mathbf{M}$ 表示因果遮罩矩阵，未来位置对应的注意力分数被置为负无穷，从而避免模型看到未来 token。多头注意力则是在多个子空间中并行执行上述注意力计算，使模型能够从不同角度捕捉上下文关系。

2.4 预训练与微调机制

预训练。预训练使用无标签文本作为数据来源，通过 next-token prediction 让模型学习通用语言模式。本项目的预训练阶段对应 TinyStories 语料，目标是让小型 MiniGPT 在有限数据和算力条件下学习基本英文短故事中的局部上下文关系。预训练结果主要通过训练损失、验证损失、perplexity 以及生成样例进行观察，具体实验

设置和结果放在第四章展开。

微调。微调是在预训练模型基础上，使用带标签或具有明确任务格式的数据继续训练，使模型适应具体任务。与预训练关注通用语言规律不同，微调更关注任务目标，例如文本分类或问答。本文后续微调实验会在预训练 MiniGPT 的基础上构建简单监督任务，相关分类头设计、损失函数和评价指标放在第五章说明。

第 3 章 小型 GPT 模型设计与实现

3.1 模型总体结构

本项目实现的是一个小参数量 Decoder-only GPT 类语言模型，其目标是在普通单机和较低算力条件下完成模型结构验证、预训练和推理生成实验。模型输入为 token id 序列，形状为 $[\text{batch_size}, \text{seq_len}]$ ；模型输出为 logits，形状为 $[\text{batch_size}, \text{seq_len}, \text{vocab_size}]$ 。其中，最后一维对应词表中每个 token 的预测分数，可用于预训练阶段的 next-token prediction 损失计算，也可用于推理阶段选择下一个 token。

从整体结构看，MiniGPT 包含词嵌入、位置嵌入、嵌入 Dropout、多层 TransformerBlock、最终 LayerNorm 以及无偏置线性输出层。该结构与第二章介绍的 GPT 基本原理一致，但参数规模明显减小，更适合课程实验中对大语言模型核心机制进行复现和验证。模型总体结构如图 3-1 所示。

图 3-1 体现了小型 GPT 的基本计算路径：离散 token id 首先被映射为连续向量表示，并与位置编码相加；随后隐藏状态经过多层因果 Transformer 模块建模上下文关系；最后通过最终 LayerNorm 和线性输出层得到词表维度上的 logits。后续预训练、文本生成和微调实验均建立在这一模型主体之上。

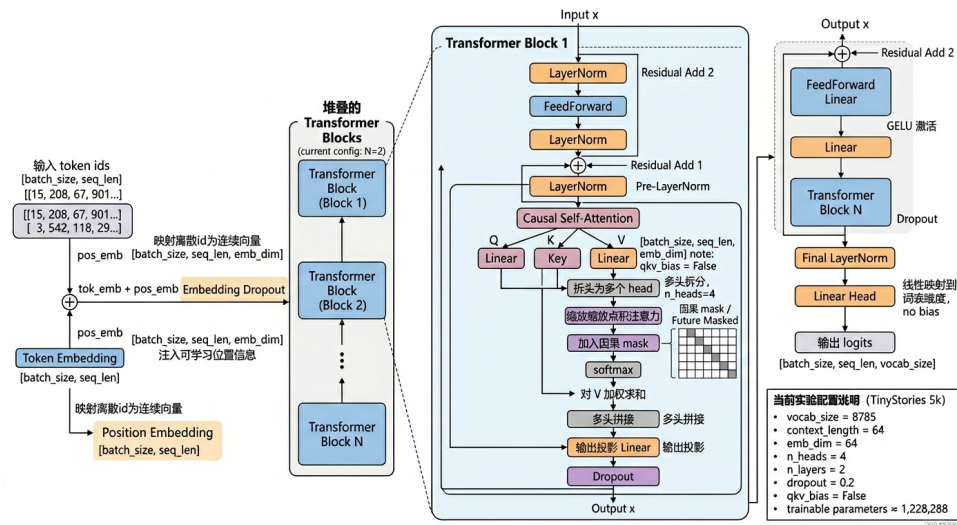


图 3-1 MiniGPT 模型总体结构图

3.2 核心模块设计

3.2.1 MiniGPTConfig 配置类

MiniGPTConfig 用于集中管理模型结构参数,包括 vocab_size、context_length、emb_dim、n_heads、n_layers、dropout 和 qkv_bias 等。这些参数分别决定词表规模、最大上下文长度、嵌入维度、注意力头数、Transformer 模块层数、正则化强度以及 Query、Key、Value 线性映射是否使用偏置。

采用配置类的好处是便于在不同实验中快速调整模型规模。例如,当训练平台算力有限时,可以减小 emb_dim、n_layers 或 context_length 来降低显存占用和训练时间;当数据规模扩大时,也可以适当增加模型容量。该设计使模型结构和实验参数之间保持清晰对应,便于后续预训练实验记录和报告分析。

3.2.2 因果多头自注意力模块

CausalSelfAttention 是 MiniGPT 中最能体现 GPT 自回归建模机制的模块。输入隐藏状态首先分别经过线性层映射为 Query、Key 和 Value,然后将嵌入维度按照注意力头数划分为多个子空间,在每个头中独立计算缩放点积注意力。多头注意力的结果重新拼接后,再经过输出投影层得到与输入维度一致的表示。

为了保证模型只能根据历史信息预测下一个 token，代码中构造了上三角因果 mask，用于屏蔽当前位置之后的未来 token。该 mask 通过 register_buffer 注册为 buffer，因此会随模型一起移动到 CPU 或 GPU，但不会作为可训练参数更新。因果注意力的核心计算可概括为：

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T + M}{\sqrt{d_k}} \right) V, \quad (3-1)$$

其中， M 表示因果遮罩矩阵。通过该机制，模型在训练时无法利用未来 token，从而与 next-token prediction 的任务目标保持一致。

3.2.3 前馈网络与 Transformer 模块

FeedForward 模块对应前馈网络（feed-forward network），采用“两层线性层 + GELU + Dropout”的结构。第一层线性层将隐藏维度从 emb_dim 扩展到 $4 * \text{emb_dim}$ ，经过 GELU 非线性激活后，再由第二层线性层映射回 emb_dim。这种先升维再降维的设计是 GPT 类 Transformer 模块中的常见结构，用于增强每个位置隐藏表示的非线性表达能力。

TransformerBlock 由因果自注意力子层和前馈网络子层组成，并采用预层归一化（Pre-LayerNorm）形式。具体来说，每个子层之前先进行 LayerNorm，然后通过对应模块计算输出，最后与原输入进行残差相加。残差连接有助于信息和梯度跨层传播，LayerNorm 能够稳定训练过程，Dropout 则用于降低小数据训练时的过拟合风险。通过堆叠多个 TransformerBlock，MiniGPT 能够逐层建立 token 之间的上下文依赖关系。

3.2.4 MiniGPT 模型主体

MiniGPT 是整个模型的主体类，内部定义了词嵌入、位置嵌入、嵌入 Dropout、多个 TransformerBlock、最终 LayerNorm 和无偏置线性输出层。在 forward 过程中，模型首先检查输入序列长度是否超过 context_length；随后根据当前序列长度自动生

成位置编号，并分别计算词嵌入和位置嵌入。二者相加后经过 Dropout，再依次输入 Transformer 模块、最终 LayerNorm 和线性输出层。

最终得到的 logits 可直接用于预训练阶段的交叉熵损失计算。推理生成时，程序则取最后一个位置的 logits，根据贪心策略或采样策略选择下一个 token，并将其追加到上下文中继续生成。因此，MiniGPT 模型主体同时服务于预训练、文本生成和后续微调实验。

3.3 模型参数配置与低算力适配

本项目并不追求训练真实大规模语言模型，而是构建一个能够体现 GPT 核心机制的小参数量 MiniGPT。为了适应单机低算力平台，实验中通过较小的 emb_dim、n_heads、n_layers、context_length 和 batch size 控制训练成本。同时，qkv_bias 默认为 False，输出层采用无偏置线性层，也有助于减少参数量。Dropout 被用于降低小数据训练中的过拟合风险。

表 3-1 小型 GPT 模型主要参数配置

参数名称	含义	设置值
vocab_size	词表大小	8785
context_length	最大上下文长度	64
emb_dim	词嵌入维度	64
n_heads	注意力头数	4
n_layers	Transformer 模块层数	2
dropout	Dropout 比例	0.2
qkv_bias	QKV 映射是否使用偏置	False
参数量	可训练参数量	1,228,288

表 3-1 中的参数来自 TinyStories 5k 预训练实验记录。可以看出，模型通过降低嵌入维度和 Transformer 层数，将可训练参数量控制在约一百二十万级别。相比真实大语言模型，该规模非常小，但已经足以展示 GPT 类模型中的词嵌入、位置嵌入、因果多头自注意力、前馈网络、残差连接和 LayerNorm 等关键机制。

代码中还提供了 `count_parameters()` 函数,用于统计模型可训练参数量,方便将不同实验配置下的模型规模写入日志和报告。`simple_test()` 则使用随机 `token ids` 测试模型输入输出形状,确认输入 `[batch_size, seq_len]` 后能够得到 `[batch_size, seq_len, vocab_size]` 的 `logits`, 从而验证模型结构搭建正确。

综上,本章完成了小型 GPT 模型架构的搭建。该模型保留了词嵌入、位置嵌入、因果多头自注意力、前馈网络、残差连接和 `LayerNorm` 等核心机制,同时通过较小参数规模适配单机低算力环境。该部分直接对应课程作业中“小参数量 GPT 类大语言模型架构搭建”的要求,并为后续预训练、推理生成和微调实验提供模型基础。

第 4 章 训练数据准备与模型预训练实验

4.1 TinyStories 数据集与文本预处理

数据集划分。本实验采用 Hugging Face 上的 TinyStories 数据集作为预训练语料。该数据集由大量简短英文故事组成,语言结构相对简单,包含常见人物、动作、对话和基础叙事关系,适合小参数量 GPT 模型学习基本英文故事生成模式。实验中选取训练集切片 `train[:5000]` 和验证集切片 `validation[:500]`, 读取字段为 `text`。这种规模既能提供较为丰富的短故事文本,又不会使 CPU 单机训练时间过长,符合低算力课程实验的目标。

文本拼接与清洗。程序读取每条故事文本后,将多条故事拼接为连续语料,并在不同故事之间加入 `<|endoftext|>` 作为分隔标记,用于表示故事边界。正则分词器将该标记作为完整特殊 `token` 处理,避免其被拆成多个无意义片段,有助于模型区分相邻故事之间的边界。文本清洗主要包括统一 Windows 和 Linux 换行、去除行尾多余空格、压缩连续空行等基础操作。为了便于重复实验,程序设置了 Hugging Face 数据缓存目录,但报告正文不展开具体本地路径,相关文件统一作为实验输出管理。

表 4-1 TinyStories 预训练数据统计

项目	训练集	验证集
数据切片	<code>train[:5000]</code>	<code>validation[:500]</code>
文本字符数	4246757	412120
token 数量	1014146	98828
样本数量	15846	1544

表 4-1 给出了预训练语料的主要统计信息。可以看出，经过分词和滑动窗口处理后，训练集和验证集均能提供足够数量的 `next-token prediction` 样本，用于观察小型 GPT 模型的训练过程和验证集表现。

4.2 分词编码与训练样本构造

分词器选择。本项目的 `tiny_tokenizer.py` 中保留了两种分词器实现：`GPT2Tokenizer` 和 `SimpleRegexTokenizer`。前者对应 GPT-2 BPE 子词分词方案，后者对应本文主实验采用的正则分词方案。两种方案的主要差异如表 4-2 所示。

表 4-2 两种分词器方案对比

对比项	GPT-2 BPE 分词器	正则分词器
代码实现	<code>GPT2Tokenizer</code>	<code>SimpleRegexTokenizer</code>
分词方式	基于 GPT-2 BPE 子词编码	按英文单词、数字、标点、换行和特殊标记切分
词表来源	使用 GPT-2 固定词表	根据训练语料构建词表
词表规模	较大	较小
泛化能力	较强，适合更复杂文本	依赖训练语料，泛化能力较弱
计算开销	输出层参数更多，训练开销更大	输出层较小，更适合低算力训练
本项目用途	作为可选方案保留	作为主实验方案使用

真实 GPT 系列模型通常更倾向于使用 BPE 或类似的子词分词方法，这类方法具有更好的开放词表能力和文本泛化能力。但本文的目标不是训练真实大规模模型，而是在低算力条件下实现小参数量 MiniGPT。若直接使用 GPT-2 BPE 分词器，会引入较大的固定词表，使词嵌入层和输出层参数量明显增加。相比之下，正则分词器能够保留较完整的英文单词，并根据 TinyStories 训练语料构建词表，词表规模更小，更适

合 CPU 单机训练和课程实验展示。因此，本文最终选择 `SimpleRegexTokenizer` 作为主实验方案，同时保留 `GPT2Tokenizer` 作为可选实现。本次预训练中，正则分词器由训练语料构建词表，最终词表大小为 8785。

滑动窗口样本构造。文本被编码为 token id 序列后，程序使用滑动窗口构造语言模型训练样本。每个输入样本为长度 `context_length` 的连续 token 序列，目标序列为输入整体右移一位后的 token 序列，即模型在每个位置都学习预测下一个 token：

$$\mathbf{x} = (x_1, x_2, \dots, x_T), \quad \mathbf{y} = (x_2, x_3, \dots, x_{T+1}). \quad (4-1)$$

本实验设置 `context_length=64`、`stride=64`。这种设置不会让相邻窗口重叠，可以减少训练样本数量和训练时间，同时仍能覆盖 `TinyStories` 中的主要文本片段。

4.3 预训练策略与程序实现

预训练任务采用 next-token prediction。MiniGPT 对每个 batch 的 logits，程序将 logits 与右移一位后的目标 token 序列计算交叉熵损失，从而同时优化序列中所有位置的下一个 token 预测误差。优化器采用 AdamW，并设置 weight decay 以提供一定正则化效果。

训练过程中，程序定期在训练集和验证集上评估 loss 与 perplexity。Perplexity 由 loss 的指数变换得到，可以理解为模型预测下一个 token 时的平均不确定性。训练脚本保存验证集 loss 最低时的 best checkpoint，同时保存最后一轮模型，并记录训练参数、metrics、loss 曲线、训练日志和文本生成样例。实验中设置 early stopping patience 为 10，本次主实验按预设的 20 个 epoch 完成。整个预训练过程在 CPU 单机环境下运行，用于验证小型模型在较低算力平台上的可运行性。

表 4-3 汇总了本次预训练的主要参数。整体配置延续了小参数量模型设计思路，通过较小的嵌入维度、层数和上下文长度控制训练成本，同时保留 GPT 类模型的核心训练流程。

表 4-3 MiniGPT 预训练参数设置

参数	设置
数据集	TinyStories
分词器	正则分词器
vocab_size	8785
context_length	64
stride	64
emb_dim	64
n_heads	4
n_layers	2
dropout	0.2
batch size	16
epochs	20
learning rate	3×10^{-4}
weight decay	0.05
训练设备	CPU
参数量	1228288

4.4 预训练结果与生成效果分析

Loss 与 perplexity 变化。预训练过程中，训练 loss 和验证 loss 均明显下降。第一次评估时，验证 loss 为 7.9660，对应验证 perplexity 为 2881.37；训练结束时，最优验证 loss 降至 3.3125，对应 perplexity 约为 27.45。这说明模型对下一个 token 的预测不确定性显著降低，已经从 TinyStories 语料中学习到一定的英文短故事语言模式。

图 4-1 展示了预训练过程中的 loss 曲线。可以看到，训练初期 loss 下降较快，随后下降速度逐渐减慢；验证 loss 与训练 loss 保持相近趋势，说明模型训练过程整体较稳定。由于模型规模较小，后期生成质量仍然有限，但 loss 曲线已经能够证明预训练流程有效运行。

表 4-4 给出了本次预训练的主要结果。最优验证 loss 出现在第 20 个 epoch、第 19700 个 step，最终训练 loss 和最终验证 loss 相差较小，说明模型虽然容量有限，但在该数据规模下没有出现特别严重的训练集与验证集分离现象。

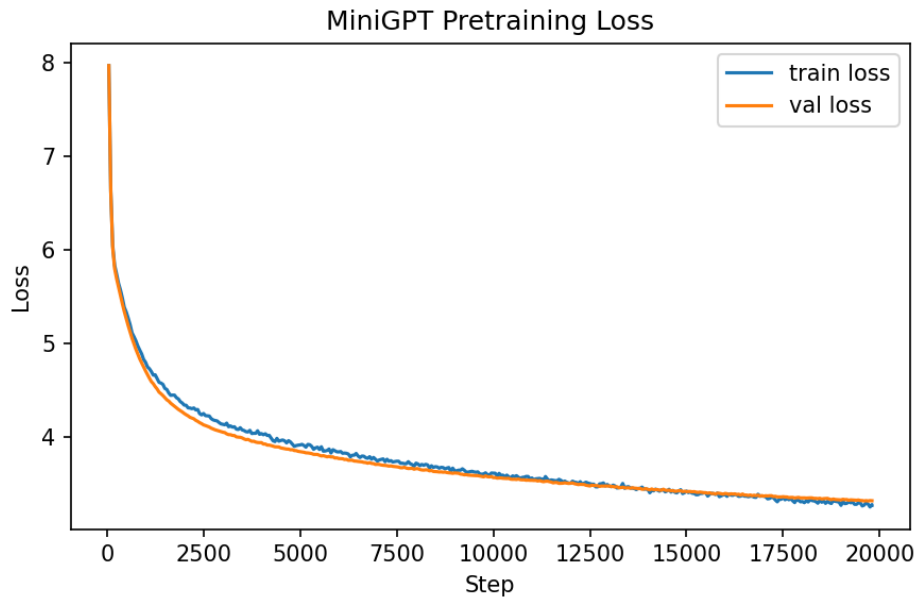


图 4-1 MiniGPT 在 TinyStories 数据集上的预训练 loss 曲线

表 4-4 预训练主要结果

指标	数值
初始验证 loss	7.9660
初始验证 perplexity	2881.37
最优验证 loss	3.3125
最优验证 perplexity	约 27.45
最优 epoch	20
最优 step	19700
最终训练 loss	3.2659
最终验证 loss	3.3147
训练耗时	约 2491 s

生成样例与效果分析。预训练完成后，使用提示词 `Once upon a time` 进行自回归生成。下面截取一段生成样例：

Prompt: `Once upon a time`

Generated: `Once upon a time, there was a little girl named Lily. She decided to her mom and sing for hours. One day, she went to the kitchen and asked her room to the man and she asked Lily got on her mommy and said,"What is so we can be your knee. You have to wait to know what I try my job you do. You can learn you fix you like to play with your parents. Now you will take another things. I wish I do you want to pick for help me." They look at least and Ben were happy. They looked for his lesson that they had lots of fun!`

从样例可以看出，模型已经能够形成较明显的故事开头和儿童故事风格，生成内容中出现人物、动作和简单对话，整体形式接近 TinyStories 的短故事叙述。然而，生成文本仍存在语法不稳定、角色关系混乱、长程一致性不足等问题，例如部分句子中动作搭配不自然，情节推进也不够连贯。这与模型规模较小、训练数据切片有限以及训练轮次有限有关。总体而言，本实验的目标不是达到真实大语言模型的生成质量，而是验证小型 GPT 预训练流程能够完整运行，并能展示出初步的语言建模效果。

本章完成了预训练部分的主要工作，包括 TinyStories 数据准备、文本拼接与清洗、分词编码、滑动窗口训练样本构造、预训练程序实现、模型预训练以及预训练效果测试与展示。这些内容对应了课程作业中“完成大模型预训练”的要求，并为后续微调实验提供了预训练模型基础。

第 5 章 微调实验与任务适应

5.1 微调任务与监督数据构建

预训练阶段的 MiniGPT 主要学习根据上下文预测下一个 token，这一目标能够让模型掌握一定的英语短故事语言模式，但尚不能直接完成明确的下游分类任务。为了考察预训练模型的任务适应能力，本文在预训练 MiniGPT 的基础上构建故事结局倾向二分类任务（story ending polarity classification），要求模型根据输入的故事结尾片段判断其结局倾向。

该任务的输入是一段 TinyStories 风格的英文故事结尾文本，输出类别为 `positive` 或 `negative`。其中，`positive` 表示故事结局积极，例如问题得到解决、人物获得帮助、冲突缓和、人物最终安全或开心；`negative` 表示故事结局消极，例如问题没有解决、人物受伤、失败、伤心、害怕、孤单，或坏结果持续存在。与普通影评或商品评论情感分类相比，该任务与预训练语料的儿童短故事风格更加一致，因此更适合

用来展示本项目中小型 GPT 模型的任务迁移过程。

监督数据由 TinyStories 风格文本构建而来。由于原始故事文本不包含分类标签，本文先抽取候选故事结尾，再依据结局倾向进行人工标注。标注时只依据当前文本中呈现的信息判断：如果困难最终被解决，则标为 **positive**；如果困难没有解决或负面结果仍然存在，则标为 **negative**；结局不清楚或难以判断的候选样本不进入最终训练集。最终主实验构建了 300 条带标签的故事结尾样本，数据格式由文本字段和标签字段组成。其中，文本字段为故事结尾片段，标签字段为人工标注的结局倾向类别。该格式既便于程序读取，也能够直接对应文本分类任务的输入和输出，具体文件路径见附录。

数据读取后，程序复用预训练阶段保存的分词器对文本进行编码，并根据模型上下文长度进行截断和填充，同时生成 **attention_mask**，用于区分真实 token 与填充位置。数据集采用固定随机种子进行分层划分，训练集、验证集和测试集数量分别为 210、45 和 45。由于测试集中正负类别样本数量并非完全相同，除准确率（**accuracy**）外，本文还报告 **precision**、**recall**、**F1**、**macro F1** 和 **weighted F1**，以更全面地衡量模型在不同类别上的表现。表 5-1 给出了微调数据集划分与标签设置。

表 5-1 微调数据集划分与标签设置

项目	设置	数量	说明
任务名称	故事结局倾向二分类	300	判断故事结尾为积极或消极
标签 0	negative	120	结局消极、问题未解决、人物受伤或难过
标签 1	positive	180	结局积极、问题解决、人物安全或获得帮助
训练集	train	210	用于参数更新
验证集	validation	45	用于选择验证集表现最优的模型
测试集	test	45	仅用于最终效果评估

5.2 微调策略与程序设计

微调模型在预训练 MiniGPT 主体后增加一个线性分类头，而不改变原有 GPT 主体结构。输入文本依次经过词嵌入、位置嵌入、Transformer 模块和最终归一化层后，得到每个 token 的隐藏状态。分类任务需要整段文本的表示，因此本文采用平均池化（mean pooling）将所有有效 token 的隐藏状态合成为文本级向量，再送入线性分类头，输出 negative 和 positive 两个类别的 logits。训练目标使用交叉熵损失。

在初始化方式上，微调程序加载预训练阶段验证集表现最优的 MiniGPT 参数，并复用预训练阶段的分词器和词表，从而保证预训练与微调阶段的 token 编码方式一致。参数更新策略上，本文最终采用全参数微调（full fine-tuning），即 MiniGPT 主体与分类头全部参与更新。这样做的原因是本项目中的 MiniGPT 规模较小，仅包含 2 层 Transformer 模块，若只训练分类头或最后一层，模型对下游分类任务的适配能力会受到限制。

训练使用 AdamW 优化器，学习率为 5×10^{-5} ，权重衰减为 0.01，batch size 为 8，训练轮数上限为 60。训练过程中根据验证集表现保存最佳模型，并在最终测试时使用该最佳模型，而不是简单使用最后一轮模型。程序记录训练损失、验证损失、accuracy、分类报告、混淆矩阵以及错误样本，用于后续分析模型在不同类别上的表现。表 5-2 汇总了主实验参数设置。

表 5-2 微调实验参数设置

参数	设置
微调数据	300 条故事结尾标注样本
初始化方式	加载预训练阶段验证集表现最优的 MiniGPT 参数
分词器	复用预训练阶段词表
参数更新方式	全参数微调，MiniGPT 主体与分类头全部参与更新
文本表示方式	平均池化（mean pooling）
最大长度	64 tokens
batch size	8
learning rate	5×10^{-5}
weight decay	0.01
epochs	60
early stopping patience	30
可训练参数量	1,228,418

5.3 微调结果与任务效果分析

最终实验结果由微调程序自动记录，包括训练曲线、测试集预测结果、分类报告和错误样本分析。图 5-1 和图 5-2 分别展示了微调过程中的 loss 与 accuracy 变化。具体图片文件路径见附录。

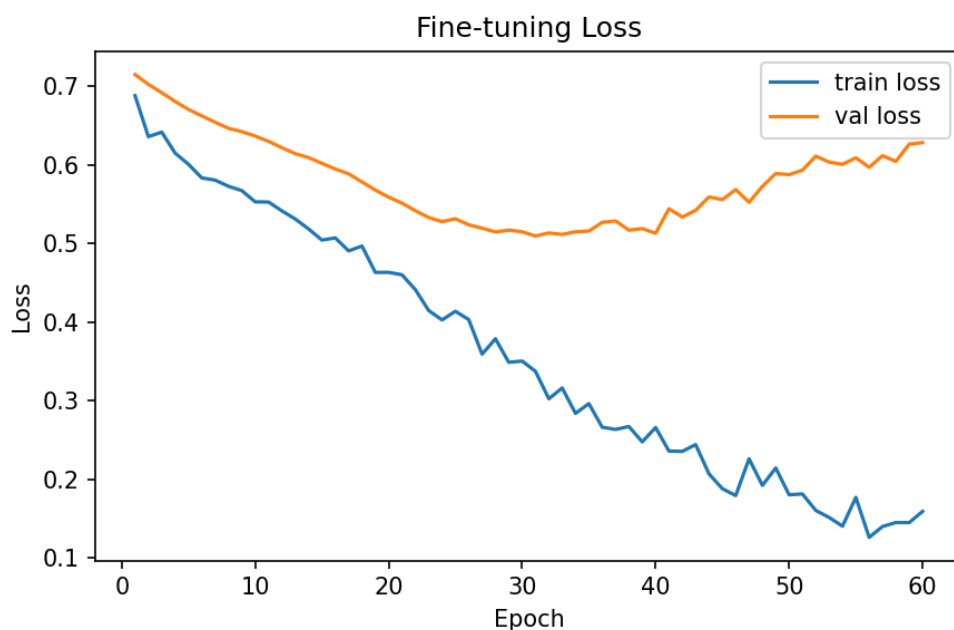


图 5-1 微调训练 loss 曲线

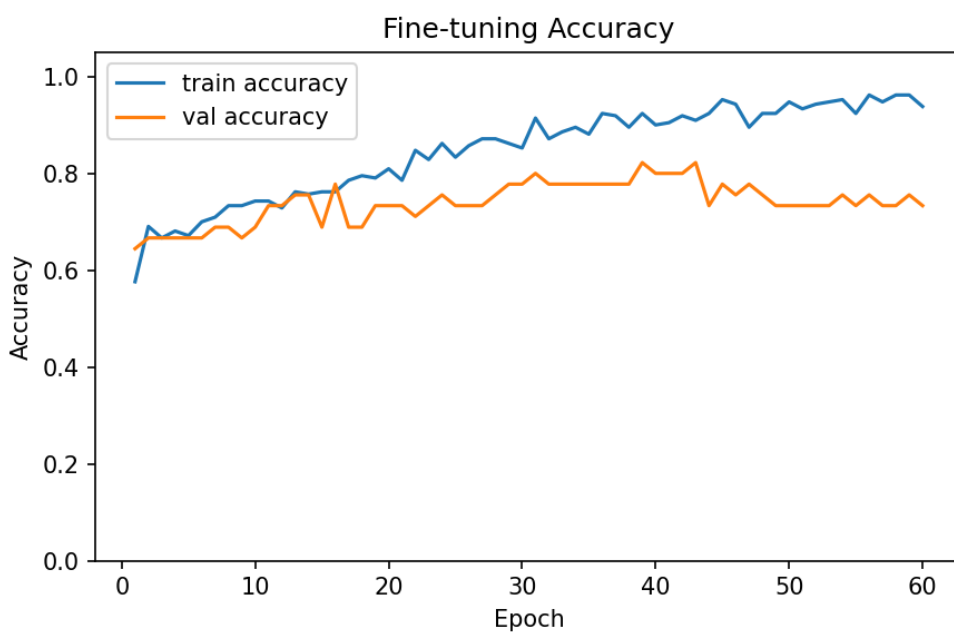


图 5-2 微调 accuracy 曲线

从训练过程看，随着 epoch 增加，训练集 loss 整体下降，训练准确率逐步升高，说明模型能够从监督数据中学习故事结局倾向的分类特征。验证集在第 39 轮达到最佳表现，验证准确率为 82.22%，验证损失为 0.5188。后续训练中，训练集准确率继

续升高，而验证集准确率和验证损失出现波动，说明模型后期存在一定过拟合趋势。因此，最终测试阶段使用第 39 轮对应的验证集最优模型进行评估。

在测试集上，模型准确率为 84.44%，macro F1 为 80.62%，weighted F1 为 83.87%。测试集分类指标如表 5-3 所示。混淆矩阵中，真实 negative 被预测为 positive 的样本有 5 条，真实 positive 被预测为 negative 的样本有 2 条，说明模型对 positive 类识别更稳定，而对部分负向结局，尤其是文本中同时包含“学到教训”等积极表达的样本，仍容易发生误判。

表 5-3 测试集分类指标

类别/指标	Precision	Recall	F1	Support
negative	81.82%	64.29%	72.00%	14
positive	85.29%	93.55%	89.23%	31
macro avg	83.56%	78.92%	80.62%	45
weighted avg	—	—	83.87%	45
overall accuracy	84.44%			
confusion matrix	tn=9, fp=5, fn=2, tp=29			

表 5-4 给出了部分测试样例及预测结果。可以看到，模型能够较好识别“成为朋友”“安全回家”“得到帮助”等积极结局，也能识别人物受伤、结局悲伤等明显负向结局。错误样例主要集中在边界文本上，例如故事中先出现负面事件，但结尾又包含“learned to be careful”一类带有积极含义的表达，模型可能过度关注“学到教训”而忽略前面更关键的负向结果。

表 5-4 测试样例与预测结果

文本摘要	真实标签	预测标签	类别概率	结果
Tom and Lucy became good friends. They played and cleaned together every day.	positive	positive	$P_{pos} = 0.933$	正确
The pup lay in his hands, still and quiet. The story ends sadly.	negative	negative	$P_{neg} = 0.799$	正确
The snake got home safe and sound after being careful in the rain.	positive	positive	$P_{pos} = 0.965$	正确
The animals were sad and missed their friend, but learned to be careful.	negative	positive	$P_{pos} = 0.905$	错误
The mouse was hurt and could not move, but learned to be careful.	negative	positive	$P_{pos} = 0.840$	错误
Timmy and the dog became best friends after going home.	positive	positive	$P_{pos} = 0.993$	正确

与预训练阶段主要用于文本续写不同，微调后的模型能够根据输入故事结尾输出明确的类别标签，说明预训练 MiniGPT 已经能够通过监督数据适配特定分类任务。这一结果表明，即使模型规模较小，只要下游数据与预训练语料在风格上较为一致，预训练模型仍可以通过简单分类头和监督微调获得可观察的任务迁移效果。

5.4 本章小结

本章围绕故事结局倾向二分类任务完成了监督微调实验。实验构建了带标签的故事结尾数据，确定了全参数微调与平均池化的分类策略，并在预训练 MiniGPT 基础上实现了分类微调程序。实验通过准确率、F1、混淆矩阵和测试样例展示了模型处理特定任务的效果，测试集准确率达到 84.44%。同时，结果也表明小模型和小规模数据仍存在局限：模型对 positive 类识别更稳定，对包含负面事件但带有积极措辞的边界样本仍容易误判，后续可通过补充负向样本和改进标注一致性进一步提升泛化

能力。

第 6 章 总结与体会

一、成果评价。

本项目围绕“从零构建大模型”的课程要求，完成了从理论学习到实验实现的完整流程。项目首先梳理了 GPT 类语言模型、分词器、Transformer、预训练与微调等关键知识点；随后基于 PyTorch 搭建了小参数量 MiniGPT 模型，并在 TinyStories 数据集上完成预训练和文本生成测试；最后，通过故事结局倾向二分类任务完成了监督微调实验。整体来看，本项目虽然没有训练真实的大规模商业模型，但在低算力条件下实现了一个可训练、可推理、可微调的小型 GPT 类模型，基本覆盖了课程任务中关于模型搭建、预训练、微调和结果分析的要求。

从实验结果看，预训练阶段模型的 loss 明显下降，生成文本也开始呈现出简单英文故事的结构，说明模型已经学习到一定的语言模式。微调阶段，模型能够根据输入故事结尾判断其倾向类别，说明预训练模型可以通过监督数据适配到特定任务中。这一过程也体现了大语言模型的一般训练思路：预训练主要学习通用语言规律，微调则用于面向具体任务进行适配。

本项目仍存在一些不足，主要原因与模型规模、数据规模和算力条件有关：

1. 模型参数量较小，生成文本的语法稳定性、情节连贯性和长程一致性仍有限；
2. 正则分词器泛化能力有限，不如成熟的 BPE 分词器；
3. CPU 训练速度较慢，限制了模型规模、训练轮数和超参数搜索范围；
4. 微调数据规模较小，部分类别边界样本仍容易误判。

后续可以从数据、模型和训练条件三个方面继续改进。首先，可以扩大预训练语料和微调数据规模，使模型接触更多语言模式和边界样本；其次，可以尝试 GPT-2

BPE 等更成熟的分词方案，并适当增加模型层数和隐藏维度；最后，如果具备 GPU 条件，可以进一步增加训练轮数，比较不同模型配置和采样策略对生成效果的影响。

二、个人体会与收获。

通过本次项目，我对大语言模型的底层流程有了更完整的认识。过去使用大模型时，我更多关注的是输入和输出效果，而这次从文本分词、token 编码、词嵌入、位置嵌入，到 Transformer 模块、loss 计算和自回归生成，逐步理解了 GPT 模型内部的基本运行方式。尤其是因果注意力机制让我更清楚地认识到，GPT 的生成过程本质上是在已有上下文条件下不断预测下一个 token。

本次实验也让我更直观地区分了预训练和微调的作用。预训练使模型学习通用语言模式，微调则让模型面向具体任务进行适配。由此也可以理解，不同 AI 工具虽然面向的任务不同，但背后往往都依赖“基础模型能力 + 任务适配”的技术思路。ChatGPT、Codex 以及其他专用 AI 工具并不需要被简单理解为完全不同的技术体系，而可以从模型能力、任务形式和交互方式的差异上进行理解。

此外，这次项目让我更加重视数据处理和训练参数的影响。分词策略会直接决定模型看到的输入形式，语料规模会影响模型能够学习到的语言模式，模型层数、上下文长度、学习率等设置也会影响训练效率和最终效果。在低算力条件下，不能盲目追求更大的模型，而需要在模型规模、数据量、训练时间和实验目标之间取得平衡。

对我个人来说，这次实践也补上了过去模型训练经验中的一部分欠缺。之前在其他任务中使用过模仿学习训练机器人仿真模型，但更多依赖现成代码和 AI 辅助，对底层结构、训练循环和评估指标理解不够。本次虽然训练对象变成了语言模型，但从数据准备、模型搭建、训练调参到结果分析的思路是相通的。通过这次较完整的实践，我对模型训练流程有了更扎实的理解，也相信这些经验能够迁移到今后的其他 AI 项目中。

参考文献与开源资源说明

1. Sebastian Raschka, *Build a Large Language Model From Scratch*, Manning, 2024. 本项目在整体学习路线、小型 GPT 模型搭建、预训练和微调流程方面参考了该书的思想。
2. LLMs-from-scratch GitHub 项目。该开源项目为本文的小型 GPT 架构实现、训练流程设计和实验组织方式提供了参考。
3. PyTorch 官方文档。本文模型搭建、损失函数、优化器、Dataset 和 DataLoader 等实现参考了 PyTorch 相关说明。
4. A. Vaswani et al., *Attention Is All You Need*, 2017. 本文关于 Transformer 和注意力机制的理论说明参考了该论文。
5. Hugging Face Datasets 官方文档。本文使用 TinyStories 数据集时，数据加载、缓存和本地保存流程参考了该文档。
6. TinyStories 数据集说明。本文预训练语料来源于 TinyStories 数据集，数据集特点和用途参考其公开说明。
7. tiktoken 分词工具相关资料。本文在分词方案设计中保留 GPT-2 BPE 分词器作为可选方案，相关实现思想参考了 tiktoken 工具说明。

A 项目文件结构

```
my_tiny_gpt/  
  
  mini_gpt.py  
  
  tiny_tokenizer.py  
  
  train_pretrain.py
```

```
generate.py

finetune_classifier.py

train_pretrain.ps1

data/

    hf_cache/

    TinyStories_train/

    TinyStories_val/

    finetune_300_labeled_detailed.csv

outputs/

    pretrain_result/

    final_result/
```

报告/

```
report.tex

pic/
```

B 主要运行命令

本项目的主要程序均可在 `my_tiny_gpt/` 目录下运行，具体训练参数已在脚本中统一配置。

```
python mini_gpt.py

python tiny_tokenizer.py

.\train_pretrain.ps1

python generate.py
```

`python finetune_classifier.py`

C 主要输出文件

预训练输出。预训练实验结果保存在 `outputs/pretrain_result/` 中，主要文件包括：

- `vocab.json`: 分词器词表与配置；
- `args.json`: 预训练运行参数；
- `metrics.json`: 预训练关键指标；
- `pretrain_loss.csv`: 训练与验证 loss 记录；
- `pretrain_loss.png`: 预训练 loss 曲线；
- `pretrain_log.txt`: 预训练日志；
- `pretrain_sample.txt`: 预训练模型生成样例；
- `pretrain_model_best.pt`: 验证集表现最优模型；
- `pretrain_model_last.pt`: 最后一轮模型。

微调输出。微调实验结果保存在 `outputs/final_result/` 中，主要文件包括：

- `args.json`: 微调运行参数；
- `metrics.json`: 微调关键指标；
- `finetune_history.csv`: 微调训练历史；
- `finetune_loss.png`: 微调 loss 曲线；

- `finetune_accuracy.png`: 微调 accuracy 曲线;
- `classification_report.txt`: 测试集分类报告;
- `finetune_predictions.csv`: 测试集预测结果;
- `finetune_errors.csv`: 错误样本记录;
- `finetune_classifier_best.pt`: 验证集表现最优分类模型;
- `finetune_classifier_last.pt`: 最后一轮分类模型;
- `finetune_sample.txt`: 微调模型测试样例。

D 实验结果截图清单

报告提交或展示时可补充以下截图材料:

- 项目目录结构截图;
- 分词器测试输出截图;
- MiniGPT 模型结构测试输出截图;
- 预训练运行过程截图;
- 预训练 loss 曲线;
- 预训练生成样例;
- 微调训练曲线;
- 微调分类报告或测试结果截图;
- GitHub 仓库文件结构截图。